

GUÍA OPENCLAW

TOOLS.md

Las manos del agente

Qué es TOOLS.md · Tools vs Skills · Las 25 herramientas · Cómo escribirlo · Mejores prácticas

I

Qué es TOOLS.md

Su rol exacto en el workspace

II

Tools vs Skills

La distinción más importante

III

Las 25 Tools

Catálogo completo con riesgos

IV

Cómo escribirlo

Estructura, ejemplos y plantillas

V

Prácticas

Qué poner, qué evitar, errores frecuentes

Módulo 5 · KeroClow · www.openagent.lat

5

MÓDULO 5

TOOLS.md

Las manos del agente

I — Qué es TOOLS.md

TOOLS.md es el archivo del workspace donde le explicás al agente los detalles específicos de tu entorno: qué skills tenés activos y para qué, qué convenciones de nombres usás, cómo querés que use cada tool en tu contexto, o cualquier información única de tu setup que no aplica a todos los usuarios.

Acá está la parte que confunde a mucha gente:

⚠ **TOOLS.md NO activa ni desactiva herramientas**

Ese es el error más común. Las herramientas reales del sistema (leer archivos, ejecutar comandos, buscar en la web, controlar el navegador) se configuran en `openclaw.json`, no acá.

TOOLS.md es pura guía en lenguaje natural — notas para el agente sobre cómo usar las herramientas en tu contexto específico. No tiene lógica técnica, no ejecuta nada, y no reemplaza la configuración de `openclaw.json`.

Lo define la documentación oficial de forma muy precisa:

📖 **Definición oficial**

"Notes about your local tools and conventions. Does not control tool availability; it is only guidance."

— docs.openclaw.ai/concepts/agent-workspace

Su lugar en el sistema de inyección

TOOLS.md se inyecta en el system prompt en el quinto lugar del orden de carga, después de AGENTS.md, SOUL.md, USER.md e IDENTITY.md. Esto significa que el agente ya sabe cómo operar (AGENTS.md), quién es (SOUL.md), con quién habla (USER.md) y tiene nombre (IDENTITY.md) antes de leer TOOLS.md.

Posición	Archivo	Qué aporta
1	AGENTS.md	Reglas de operación y comportamiento
2	SOUL.md	Personalidad, tono y límites

3	USER.md	Quién es el usuario y cómo tratarlo
4	IDENTITY.md	Nombre, emoji y vibra del agente
5 ← acá	TOOLS.md	Notas sobre el entorno y las herramientas disponibles
6	HEARTBEAT.md	Checklist para el heartbeat



¿Cuándo importa TOOLS.md?

Importa cuando tu setup tiene algo que el agente no puede saber por defecto. En un bot simple de respuestas, TOOLS.md puede estar casi vacío. En un setup con varios skills activos, automatizaciones cron, o convenciones específicas de carpetas y formatos, TOOLS.md es lo que diferencia un agente que entiende tu entorno de uno que siempre pregunta.

II — La distinción más importante: Tools vs Skills

Antes de entender TOOLS.md, hay que tener clara la diferencia entre Tools y Skills. Son dos conceptos completamente distintos en OpenClaw, pero se confunden todo el tiempo porque ambos tienen que ver con 'lo que el agente puede hacer'.

 Tools	 Skills
Son órganos Determinan si el agente puede hacer algo. Sin tool habilitada, no hay acción posible — como tener manos o no tenerlas.	Son manuales Le enseñan al agente cómo combinar tools para lograr tareas. Sin tool habilitada, el skill no sirve de nada.
<i>Ejemplos: read, write, exec, web_search, browser, memory_search, message, cron</i>	<i>Ejemplos: gog (Google Workspace), github, obsidian, slack, openai-whisper-api, notion</i>
Se configuran en: <code>openclaw.json</code>	Se instalan con: <code>clawhub install <skill></code>

La pregunta clave

Instalar un skill de Obsidian le enseña al agente a organizar notas. Pero sin la tool 'write' habilitada en openclaw.json, no puede escribir un solo archivo. El skill sabe qué hacer. La tool decide si puede hacerlo.

¿Y TOOLS.md dónde encaja?

TOOLS.md está en un tercer nivel distinto: no controla si una tool está habilitada (eso es openclaw.json), ni le enseña al agente a usar un servicio (eso es un skill). TOOLS.md le dice al agente los detalles únicos de tu setup para que pueda usar las tools que ya tiene habilitadas de forma más eficaz.

Nivel	Dónde	Para qué
1 — Habilitar	openclaw.json	Activar o desactivar tools: read, exec, browser, etc.
2 — Enseñar	Skills (~/openclaw/workspace/skills/)	Cómo usar Google Drive, GitHub, Obsidian, Whisper...
3 — Contextualizar	TOOLS.md	Qué skills activos tenés, qué convenciones usás, cómo usar las tools en tu contexto

III — Las 25 Tools: catálogo completo

OpenClaw tiene 25 tools nativas. La documentación oficial documenta 18 — las 7 restantes se encuentran en el código fuente e incluyen herramientas de sesiones, multi-agente y un motor de workflows. Acá están todas, organizadas por nivel de riesgo y función.

GRUPO 1 — Herramientas de sistema y web (8 tools)

Estas 8 tools cubren acceso a archivos, ejecución de comandos y navegación web. Son las más comunes en setups de uso general y desarrollo, pero no son obligatorias para todos los casos. Un bot de WhatsApp de atención al cliente, por ejemplo, funciona perfectamente sin ninguna de estas — usa el perfil messaging, que solo necesita message y sessions.

read	Bajo	Lee archivos del workspace y sistema de archivos.	Capa 1
write	Medio	Crea y sobrescribe archivos completos. <i>Puede reemplazar archivos existentes sin confirmación.</i>	Capa 1
edit	Medio	Edita archivos de forma estructurada, buscando y reemplazando bloques específicos.	Capa 1
apply_patch	Medio	Aplica parches multi-bloque en uno o varios archivos simultáneamente. <i>Solo disponible en modelos OpenAI.</i>	Capa 1
exec	Muy alto	Ejecuta cualquier comando de shell en el workspace. <i>La tool más poderosa y peligrosa. Recomendado activar approvals para exec.</i>	Capa 1
process	Medio	Gestiona procesos en segundo plano: listar, monitorear, terminar. <i>Casi siempre se activa junto a exec.</i>	Capa 1
web_search	Bajo	Búsqueda web mediante proveedor externo (Brave, Gemini, Perplexity, Grok). <i>Requiere API key del proveedor configurada en openclaw.json.</i>	Capa 1
web_fetch	Medio	Descarga y extrae el contenido de una URL como texto/markdown. No ejecuta JavaScript. <i>Habilitado por defecto. Para páginas JS-heavy, usar Firecrawl como fallback.</i>	Capa 1

CAPA 2 — Capacidades avanzadas (17 tools): activar según necesidad

Estas tools convierten al agente de respondedor a asistente real. Cada una expande la superficie de riesgo — evalúa si el caso de uso justifica habilitarlas.

Control de navegador y visuales

browser	Alto	Controla Chrome: navega, hace clic, completa formularios, toma capturas. <i>Ver configuración en openclaw.json > browser.</i>	Capa 2
canvas	Bajo	Workspace visual para el agente: diagramas, interfaces A2UI, widgets.	Capa 2
image	Bajo	Permite al agente analizar e interpretar imágenes enviadas en la conversación.	Capa 2

Memoria entre sesiones

memory_search	Medio	Busca en la memoria vectorial del agente por semántica (~400 tokens por chunk). <i>Proveedores: GGUF local → OpenAI → Gemini → Voyage (auto-selección).</i>	Capa 2
memory_get	Medio	Recupera entradas específicas de la memoria persistente.	Capa 2

Multi-sesión

sessions_list	Bajo	Lista todas las sesiones activas.	Capa 2
sessions_history	Medio	Historial de conversación de una sesión específica.	Capa 2
sessions_send	Alto	Envía mensajes entre sesiones paralelas.	Capa 2
sessions_spawn	Alto	Crea subagentes para tareas paralelas independientes.	Capa 2
session_status	Bajo	Consulta el estado de una sesión.	Capa 2

Mensajería, automatización y hardware

message	Muy alto	Envía mensajes a WhatsApp, Telegram, Slack, Discord, iMessage. <i>Recomendado: solo usarlo para mensajes hacia uno mismo, nunca hacia terceros.</i>	Capa 2
nodes	Muy alto	Control de hardware en dispositivos emparejados: cámara, GPS, captura de pantalla. <i>Pensar dos veces antes de activar: ¿cuándo necesitas que la IA abra tu cámara sola?</i>	Capa 2

cron	Alto	Tareas programadas: diarias, horarias, con expresión cron. <i>Combinado con message es el motor de automatización de OpenClaw.</i>	Capa 2
gateway	Alto	Permite al agente reiniciarse a sí mismo.	Capa 2
agents_list	Bajo	Lista los IDs de agentes disponibles en setups multi-agente. <i>No necesario en setups de un solo agente.</i>	Capa 2

Motor de workflows (avanzado, no documentado oficialmente)

llm_task	Medio	Inserta un paso de procesamiento LLM dentro de un workflow multi-paso. <i>Solo útil si usás lobster como motor de workflows.</i>	Capa 2
lobster	Medio	Motor de workflows para definir procesos multi-paso con lógica condicional. <i>No documentado en docs oficiales. Encontrado en código fuente.</i>	Capa 2

IV — Cómo escribir TOOLS.md

TOOLS.md es un archivo Markdown libre — no tiene estructura obligatoria. La documentación oficial ni siquiera impone un formato. Eso le da flexibilidad, pero también significa que hay que saber qué poner para que sea útil.

El archivo que genera OpenClaw por defecto

Cuando corrés openclaw setup por primera vez, el TOOLS.md generado es casi vacío — solo un título y algunas secciones de ejemplo. Eso es intencional: el archivo está diseñado para que vos lo completes con lo que es único de tu setup.

La clave del diseño

'Skills define cómo funcionan las tools. Este archivo es para tus especificidades — lo que es único de tu setup.' — Documentación oficial de OpenClaw.

Skills = instrucciones generales que aplican a cualquier usuario. TOOLS.md = el contexto específico de tu entorno que solo vos conocés.

Qué tiene sentido poner

No hay estructura obligatoria. Pero hay ciertos tipos de información que realmente ayudan al agente a operar sin preguntar cosas que debería saber:

Línea del archivo	Qué hace / Qué significa
# TOOLS.md	
## Skills activos y para qué - openai-whisper-api → transcribir mensajes de voz entrantes - web_fetch → consultar APIs externas del negocio	Recordatorio de qué tiene instalado y para qué sirve
## Convenciones del workspace - Logs: /workspace/logs/YYYY-MM-DD.md - Formato fecha: YYYY-MM-DD - Formato registro: "nombre canal tipo-consulta"	Estructura de carpetas y formato de archivos
## Notas de uso de tools - browser: solo navegar en dominios autorizados - web_fetch: preferir formato markdown - exec: confirmar antes de borrar archivos	Instrucciones específicas para tu contexto
## TTS (si usás voz) - Voz preferida: "Alloy" (OpenAI TTS) - Canal de entrega: Telegram personal	Solo si tenés el skill de voz activo

Tres ejemplos por tipo de bot

Cada setup tiene sus propias especificidades. Estos ejemplos muestran qué tipo de información vale la pena anotar según el caso de uso:

Bot de atención al cliente (WhatsApp / Telegram)

Línea del archivo	Qué hace / Qué significa
## Skills activos - openai-whisper-api → transcribir mensajes de voz entrantes - web_fetch → consultar APIs externas del negocio	
## Convenciones del workspace - Logs diarios: /workspace/logs/YYYY-MM-DD.md - Formato contacto: "nombre canal consulta"	
## Límites del browser - Solo navegar dominios autorizados del negocio - Nunca completar formularios de pago ni login externo	

Bot de soporte técnico (Telegram/Slack)

Línea del archivo	Qué hace / Qué significa
## Skills activos - github → consultar issues, PRs y estado de CI/CD - session-logs → buscar en historial de sesiones	
## Convenciones del repo - Branches: feat/, fix/, chore/ - main siempre protegida – no push directo	
## exec - Confirmar con el usuario antes de cualquier rm - No instalar paquetes globales sin avisar	

Agente personal con automatizaciones (cron)

Línea del archivo	Qué hace / Qué significa
## Automatizaciones activas - 07:00 → briefing diario → Telegram - 20:00 → resumen de actividad del día → Telegram	
## TTS - Skill: openai-whisper-api (transcripción entrante) - Voz saliente: "Alloy" via ElevenLabs	
## message - Solo enviar a canales propios (nunca a clientes directo)	

- Canal prioritario: Telegram @canal-personal

V — Mejores prácticas, errores frecuentes y checklist

Qué poner — y qué no poner

✓ Sí incluir	✗ No incluir
Skills activos y para qué los usás	API keys o contraseñas
Convenciones de naming de tu proyecto	Instrucciones de comportamiento general (eso va en AGENTS.md)
Paths a ejecutables no estándar	Personalidad o tono (eso va en SOUL.md)
Reglas de uso para exec o browser en tu contexto	Información sobre el usuario (eso va en USER.md)
Formatos de fecha o log que usás	Instrucciones que aplican a todos los setups de OpenClaw
Qué skills están instalados y para qué	Bloques de código largos o documentación completa de APIs

Los 3 errores más comunes

Error 1: Poner la configuración de tools en TOOLS.md

TOOLS.md no controla qué tools están habilitadas. Si querés activar exec o browser, eso se configura en la sección tools de openclaw.json. Escribir 'exec: habilitado' en TOOLS.md no hace nada técnico — es como anotar 'el auto arranca solo' sin girar la llave.

Error 2: Duplicar contenido de AGENTS.md

AGENTS.md ya define cómo opera el agente. TOOLS.md no debería repetir reglas de comportamiento, protocolos de atención, ni flujos de conversación. Si algo aplica a cómo el agente se comporta, va en AGENTS.md. Si aplica a cómo usa las herramientas en tu entorno específico, va en TOOLS.md.

Error 3: Hacerlo demasiado largo

TOOLS.md se inyecta en cada sesión y consume tokens. La documentación oficial recomienda mantenerlo corto. El límite de bootstrapMaxChars es 20.000 caracteres, pero si TOOLS.md llega a eso, algo está mal. El objetivo es ser un mapa de referencia rápida, no una documentación exhaustiva.

Checklist antes de guardar tu **TOOLS.md**

✓ Antes de guardar

- ¿Tiene información única de tu entorno que el agente no puede adivinar?
 - ¿Está libre de API keys, contraseñas o tokens?
 - ¿Tiene menos de 2-3 páginas de contenido útil?
 - ¿Las instrucciones de comportamiento ya fueron a AGENTS.md?
 - ¿La personalidad y tono ya fueron a SOUL.md?
 - ¿Los datos del usuario ya fueron a USER.md?
- Si respondés sí a todas, el archivo está bien.

TOOLS.md en producción: automatización con cron + message

El caso de uso más poderoso de documentar bien las tools en **TOOLS.md** es cuando usás automatizaciones con cron. Cuando el agente tiene que ejecutar tareas programadas sin que vos estés ahí para guiarlo, necesita encontrar la información de contexto en el archivo — no puede preguntarte.

💡 Ejemplo real: briefing diario automatizado

El agente tiene una tarea cron que corre a las 7:00. Sin **TOOLS.md** bien escrito, tiene que adivinar a qué speaker mandar el TTS, en qué formato quieras los logs, o qué canal de Telegram usar.

Con **TOOLS.md**: 'Speaker por defecto: HomePod cocina. Canal Telegram: @mi-canal-personal. Logs en /workspace/logs/YYYY-MM-DD.md.' El agente ejecuta sin preguntas.

VI — Hoja de referencia rápida

Tool	Capa	Función	Riesgo	Nota clave
read	1	Leer archivos	Bajo	—
write	1	Escribir archivos	Medio	Sobreescribe sin confirmar
edit	1	Edición estructurada	Medio	—
apply_patch	1	Parches multi-bloque	Medio	Solo modelos OpenAI
exec	1	Comandos de shell	Muy alto	Activar approvals
process	1	Gestión de procesos	Medio	Úsalo con exec
web_search	1	Búsqueda web	Bajo	Necesita API key
web_fetch	1	Leer URLs	Medio	No ejecuta JS
browser	2	Control de Chrome	Alto	Ver config en openclaw.json
canvas	2	Workspace visual	Bajo	—
image	2	Analizar imágenes	Bajo	—
memory_search	2	Búsqueda semántica en memoria	Medio	—
memory_get	2	Recuperar memoria	Medio	—
sessions_list	2	Listar sesiones	Bajo	—
sessions_history	2	Historial de sesión	Medio	—
sessions_send	2	Mensajes entre sesiones	Alto	—
sessions_spawn	2	Crear subagentes	Alto	—
session_statuses	2	Estado de sesión	Bajo	—
message	2	Enviar mensajes externos	Muy alto	Solo hacia vos mismo
nodes	2	Control de hardware	Muy alto	Pensar bien antes de activar
cron	2	Tareas programadas	Alto	Motor de automatización
gateway	2	Reiniciar el agente	Alto	—
agents_list	2	Listar agentes	Bajo	Multi-agente únicamente
llm_task	Ext	Paso LLM en workflow	Medio	Solo con lobster
lobster	Ext	Motor de workflows	Medio	No documentado oficialmente

→ Siguiendo módulo: USER.md + IDENTITY.md

TOOLS.md define las herramientas del entorno. El siguiente paso es USER.md e IDENTITY.md — los dos archivos que completan la relación entre el agente, el operador y el usuario final. USER.md define quién es la persona con la que el agente interactúa. IDENTITY.md define el

nombre, emoji y vibra del propio agente.